

Camada de Aplicação e Transporte

Aluna

Rielly Luiza Duarte da Silva

1 Estratégia Adotada

A ideia principal foi criar um **"controle de recibos"** no próprio programa, já que o UDP sozinho não garante que a mensagem chegou.

- Toda mensagem enviada ganha um ID sequencial (1, 2, 3...) e é guardada num dicionário chamado `pending_messages`, no formato `id: texto`. Enquanto o ID dela estiver nesse dicionário, ela é considerada "pendente", ou seja, ainda não tem a certeza que chegou.
- Como o programa tem duas coisas rodando ao mesmo tempo (o usuário digitando mensagens e uma thread separada esperando as respostas), existe risco das duas mexerem no dicionário `pending_messages` ao mesmo tempo e dar errado. Pra evitar isso, existe o **lock**: sempre que alguma parte do código vai mexer no dicionário, ela "trava" o acesso, faz a alteração, e depois libera. Assim só um mexe por vez.
- A thread separada `listen_receipts` fica o tempo todo escutando se chegou algum pacote `DELIVERED—jIDj`. Quando chega, ela pega o ID confirmado, remove esse id de `pending_messages` e mostra na tela que a mensagem foi entregue. Ela roda em paralelo justamente pra não travar o terminal esperando resposta, ent o usuário pode continuar digitando mensagens novas normalmente enquanto isso acontece em segundo plano.

Já os comandos `/status` e `/reenviar` são formas do usuário interagir manualmente com esse controle:

- **/status**: só olha o que ainda tá dentro de `pending_messages` e mostra na tela, sem enviar nada
- **/reenviar**: percorre tudo que ainda tá pendente e manda de novo pro servidor, pra tentar enviar as que foram perdidas na rede

Resumindo: a gente não confia que a mensagem chegou só porque foi enviada, só assume que ta "entregue" quando um recibo confirma isso, e enquanto não confirma, ela fica marcada como pendente até o usuário decidir reenviar.

2 Captura de tela 1

```
sex 11 set 16:40
luiza@luiza-IdeaPad-Slim-3-15IRH10: ~/Faculdade/SPO$ python3 chat_receiver.py
Online na porta 5001 (Drop Rate: 40.0%)...
[CANAL] Pacote descartado artificialmente!
Mensagem recebida ID 2 de ('127.0.0.1', 45914): isso é um teste
[CANAL] Pacote descartado artificialmente!
Mensagem recebida ID 4 de ('127.0.0.1', 45914): teste1
[CANAL] Pacote descartado artificialmente!
[CANAL] Pacote descartado artificialmente!
Mensagem recebida ID 7 de ('127.0.0.1', 45914): teste4
Mensagem recebida ID 8 de ('127.0.0.1', 45914): teste5
[]

luiza@luiza-IdeaPad-Slim-3-15IRH10: ~/Faculdade/SPO$ python3 chat_sender.py
Comandos especiais:
/status      Mostra mensagens ainda pendentes
/reenviar    Reenvia todas as mensagens pendentes

Digite uma mensagem: oi!
/enviar: msg 1 enviada, aguardando confirmação...
Digite uma mensagem: isso é um teste
/enviar: msg 2 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 2 confirmada: "isso é um teste"
Digite uma mensagem: disciplina: redes de computadores
/enviar: msg 3 enviada, aguardando confirmação...
Digite uma mensagem: teste1
/enviar: msg 4 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 4 confirmada: "teste1"
Digite uma mensagem: teste2
/enviar: msg 5 enviada, aguardando confirmação...
Digite uma mensagem: teste3
/enviar: msg 6 enviada, aguardando confirmação...
Digite uma mensagem: teste4
/enviar: msg 7 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 7 confirmada: "teste4"
Digite uma mensagem: teste5
/enviar: msg 8 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 8 confirmada: "teste5"
Digite uma mensagem:
// Mensagem 8 confirmada: "teste5"
```

3 Captura de tela 2

```
sex 11 set 16:42
luiza@luiza-IdeaPad-Slim-3-15IRH10: ~/Faculdade/SPO$
Mensagem recebida ID 2 de ('127.0.0.1', 45914): isso é um teste
[CANAL] Pacote descartado artificialmente!
Mensagem recebida ID 4 de ('127.0.0.1', 45914): teste1
[CANAL] Pacote descartado artificialmente!
[CANAL] Pacote descartado artificialmente!
Mensagem recebida ID 7 de ('127.0.0.1', 45914): teste4
Mensagem recebida ID 8 de ('127.0.0.1', 45914): teste5
Mensagem recebida ID 9 de ('127.0.0.1', 45914): /reenviar
Mensagem recebida ID 1 de ('127.0.0.1', 45914): oi!
Mensagem recebida ID 3 de ('127.0.0.1', 45914): disciplina: redes de computadores
Mensagem recebida ID 5 de ('127.0.0.1', 45914): teste2
[CANAL] Pacote descartado artificialmente!
Mensagem recebida ID 6 de ('127.0.0.1', 45914): teste3
[]

luiza@luiza-IdeaPad-Slim-3-15IRH10: ~/Faculdade/SPO$
/enviar: msg 6 enviada, aguardando confirmação...
Digite uma mensagem: teste4
/enviar: msg 7 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 7 confirmada: "teste4"
Digite uma mensagem: teste5
/enviar: msg 8 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 8 confirmada: "teste5"
Digite uma mensagem: /reenviar
/enviar: msg 9 enviada, aguardando confirmação...
Digite uma mensagem:
// Mensagem 9 confirmada: "/reenviar"
Digite uma mensagem: /reenviar
/reenviar: ID 1: "oi!"
/reenviar: ID 3: "disciplina: redes de computadores"
/reenviar: ID 5: "teste2"
/reenviar: ID 6: "teste3"
Digite uma mensagem:
// Mensagem 1 confirmada: "oi!"
Digite uma mensagem:
// Mensagem 3 confirmada: "disciplina: redes de computadores"
Digite uma mensagem:
// Mensagem 5 confirmada: "teste2"
Digite uma mensagem: /reenviar
/reenviar: ID 6: "teste3"
Digite uma mensagem:
// Mensagem 6 confirmada: "teste3"
Digite uma mensagem: /reenviar
/reenviar: Nenhuma mensagem pendente para reenviar.
Digite uma mensagem:
```

4 Controle na camada de aplicação VS TCP

Na camada de aplicação, toda a responsabilidade de garantir a entrega fica **por conta do dev**: criar ID, guardar pendências, esperar confirmação e reenviar quando necessário. Isso dá mais controle sobre como cada coisa funciona, mas também dá mais trabalho e mais chance de erro

Com TCP, isso tudo já vem pronto. **O próprio protocolo garante** que os dados cheguem (na ordem certa também), e refaz o envio sozinho se algo se perder, sem a aplicação precisar saber que isso aconteceu. É mais simples de usar, mas em troca a aplicação tem menos controle sobre esse processo (e normalmente é um pouco mais lento por causa desse controle extra que o TCP faz por trás)